## Slides

**1. Title**
"My project asks how much of a House candidate's vote share of campaign money can explain. Money and votes are clearly connected, but not in a straight line. Party, incumbency and what your opponents raise all matter, and they interact. So I used machine learning to learn those patterns from the data."

**2. Data and approach**
"I combined two public datasets: FEC campaign finance for the money, and MIT's Election Lab for the official vote counts. They don't share an ID, so I matched them by district and last name, handling accents, two-word last names, and even names the FEC entered backwards. That left 1,023 candidates in 405 races. The features are money compared with opponents, plus party, incumbency and number of candidates. I split the data by district so the same race is never in both training and testing, compared three models, and chose the random forest."

**3. What I found**
"Money and votes move together, but money is much more lopsided than votes. Party sets a floor and a ceiling: even major-party candidates with almost no money often got 30 to 45 percent. Incumbents raise the most and win the most. The model is about 4 points off on races it had never seen.

Its limits: close races, where it called 6 of 9; same-party races; and this is correlation, not causation, because donors give to candidates who already look likely to win."

**4. From notebook to production**
"The top row is my path from notebook to Heroku, with GitHub Actions testing every change. The bottom row is the main idea: I packaged the app once as a Docker container, and that same container runs in five places. Let me show you each one."

## Demo

**1. Writeup (GitHub repo)**
"Everything lives in this repository. The README is my writeup: the problem, where the data comes from, the features, the results, and a troubleshooting log of every problem I hit during deployment."

**2. Notebook**
"Here's the notebook. I cleaned and matched the data step by step, with the counts after each step. This chart is the money-vs-votes relationship. Here I compared three models with cross-validation, and here's the final test: about 3.8 points of error on races the model never saw."

**3. Flask (localhost)**
"This is the model in a Flask app running on my computer, at localhost. I can pick any 2024 race and see predicted versus actual vote share." *(Pick a race, then click Edit as a what-if and change the money.)* "And I can change a candidate's money to see what the model thinks would happen." *(Open AK-0.)* "Here's one it got wrong. Alaska had two Democrats, and the model gave the second one 21 percent when he actually got 1. I found this by using my own app. I tested a fix, it didn't help, so the app now warns users on races like this."

**4. CI/CD (GitHub Actions)**
"This workflow file is my CI/CD pipeline. Every time I push to GitHub, it installs the packages, tests that the app loads and its pages respond, and only then deploys to Heroku." *(Switch to the Actions tab.)*

**5. Heroku**
"And here's the result: the same app, live on the web, deployed automatically by that pipeline."

**6. Docker Hub**
"Next, I packaged the app as a Docker container. This Dockerfile starts from Python, installs the packages and starts the app." *(Switch to Docker Hub.)* "I pushed the image to Docker Hub, so anyone can pull it." *(Switch to localhost:8081.)* "And here it is running on my machine, straight from that image."

**7. Heroku container**
"This second workflow deploys the container version. It builds the image, starts it, checks that it responds, then deploys." *(Open the app.)* "It looks the same as before, but here Heroku is running my Docker container instead of setting up Python itself."

**8. AWS EC2**
"Here's my server on AWS EC2, running, with its public IP address." *(Open the app.)* "I installed Docker on it and ran the same container from Docker Hub. The hardest part was a timeout: I tested on the server, then from my computer, and that pointed to the firewall, which needed a rule to allow web traffic."

**9. AWS SageMaker**
"SageMaker can't pull from Docker Hub, so I pushed the same image to Amazon's own registry, ECR. Here's the endpoint: InService and serverless, so it only costs money while it's predicting." *(Show deploy.py.)* "This script creates the endpoint and sends a sample race." *(Run it.)* "SageMaker returns the prediction as data: about 64 to 36, rated Safe Democrat. One problem here was that SageMaker rejected my image's format at first, so I rebuilt it in the format it requires."

**10. Kubernetes**
"Finally, Kubernetes. This configuration file runs two copies of the app and checks their health through the ping route." *(Run kubectl get pods.)* "Here are both copies, running." *(Show the dashboard.)* "And the dashboard view. When I updated the app this morning, Kubernetes

started the new copies before removing the old ones, so the app never went down." *(Open 127.0.0.1:63146.)* "And here's the app, served from Kubernetes."

## Close (back on slide 4)

"The hardest problems taught me the most: the EC2 firewall, SageMaker's image format, and finding a flaw in my own model by actually using the app. Next, I'd test it against the 2026 midterms as new ground truth, and add more election years. My takeaway: money explains a lot of the vote, but not the close races, and those are the ones that matter. Thank you."